

Отчет по соревнованию ASR 2026: Spoken Numbers Recognition Challenge

1. Постановка задачи и требования

Цель проекта — построить компактную систему автоматического распознавания речи для русских числительных в диапазоне от 1000 до 999999. По условию соревнования модель должна:

- работать с входным аудио 16 кГц;
- содержать не более 5 миллионов параметров;
- обучаться с нуля только на предоставленном `train`-сплите;
- не использовать `dev` для дообучения;
- выдавать в `submission` только целые числа в цифровой записи;
- оптимизироваться под CER, а на Kaggle итоговый рейтинг считается по `harmonic mean CER` для `in-domain` и `out-of-domain speakers`.

Kaggle team name: Trifonov Rezitdinov

2. Данные

В локальной разработке использовались два сплита:

- `train`: 12 553 примера, 6 спикеров;
- `dev`: 2 265 примеров, 10 спикеров.

Краткая статистика по данным:

- `train`: только `wav`, частоты дискретизации 22.05 и 24 кГц;

- dev: wav и mp3, все записи приведены к 16 кГц на стороне датасета;
- пол в train: 7665 female / 4888 male;
- пол в dev: 988 female / 1277 male.

Так как валидационный и тестовый наборы содержат unseen speakers и шумные записи, основной акцент был сделан на устойчивость к смене говорящего, артефактам кодирования и небольшим искажениям сигнала.

3. Подход

В проекте реализована end-to-end ASR-модель на базе Conformer + CTC. Общий конвейер:

```
audio -> log-mel spectrogram -> Conformer encoder -> CTC  
decoding -> text -> integer
```

Ключевая идея решения состоит в том, что модель предсказывает не цифры напрямую, а русскую словесную форму числа, после чего отдельный постпроцессор преобразует ее обратно в целое число.

Такой вариант был выбран по двум причинам:

- в аудио произносится именно словесная форма числа;
- текстовая нормализация позволяет упростить постановку задачи и использовать компактный посимвольный CTC-словарь.

4. Архитектура модели

Итоговая модель — небольшой Conformer-CTC энкодер:

- вход: log-mel спектрограмма, 80 mel bins;
- `n_fft = 400`, `hop_length = 160`, то есть окно 25 мс и шаг 10 мс;
- двухслойный Conv2D subsampling со stride 2 и итоговым уменьшением временной оси в 4 раза;
- 8 Conformer-блоков;
- `d_model = 144`;

- `n_heads = 4`;
- сверточное ядро в Conformer `kernel_size = 31`;
- CTC-head поверх выходов энкодера.

Число параметров по сохраненному конфигу: `4 492 248`, что удовлетворяет ограничению `<= 5M`.

5. Текстовое представление и декодирование

Таргеты формируются автоматически:

1. integer label из CSV;
2. преобразование в словесную русскую форму через `num2words`;
3. посимвольное кодирование в CTC-словарь;
4. декодирование обратно в строку;
5. парсинг строки в число через `words_to_num()`.

Словарь строится из набора слов, используемых для записи русских числительных:

- единицы;
- числа 10-19;
- десятки;
- сотни;
- формы слова `тысяча`.

В сохраненном конфиге размер словаря равен `24` классам вместе с `blank` для CTC.

Во время обучения используется `greedy decoding` для быстрой валидации. В `train.py` также предусмотрена финальная `beam-search` оценка на лучшем чекпойнте, в сохраненных inference-артефактах для `dev` зафиксирован итоговый CER `0.1496`.

6. Предобработка и аугментации

Аудио загружается через `torchaudio` и приводится к единому формату:

- `mono`;
- `sample rate 16 кГц`;
- `log-mel` признаки.

Для улучшения обобщающей способности использовались следующие аугментации:

- SpecAugment:
 - 2 частотные маски, `freq_mask_param = 15`;
 - 2 временные маски, `time_mask_param = 35`;
- случайное временное растяжение/сжатие в диапазоне `+−5%`;
- опциональное смешивание с шумом при наличии внешней директории `noise_dir`, вероятность применения `0.4`, случайный SNR от 5 до 20 dB.

Дополнительно вычисляются `mean/std` по `mel`-признакам `train`-сплита и применяются для нормализации `train` и `dev`.

7. Обучение

Обучение выполнено в `src/train.py` со следующими настройками:

- `optimizer: AdamW`;
- `learning rate: 3e-4`;
- `weight decay: 1e-2`;
- `batch size: 32`;
- `число эпох: 60`;
- `gradient clipping: 5.0`;
- `loss: CTCLoss(blank=0, zero_infinity=True)`;

- scheduler: cosine decay с опциональным warmup;
- mixed precision используется на CUDA, если доступна.

Модель сохраняет:

- `best.pt` — лучший чекпойнт по CER на `dev`;
- `last.pt` — последний чекпойнт;
- `config.json`;
- `norm_stats.pt`;
- `history.json`.

8. Результаты

8.1. Динамика обучения

По `history.json` обучение сходилось стабильно:

- epoch 1: `loss = 1.578`, `CER = 0.973`;
- epoch 18: `CER = 0.1839`;
- epoch 41: `CER = 0.014`;
- лучший результат получен на epoch 59: `loss = 0.0074`, `CER = 0.1504`;
- epoch 60: `loss = 0.0076`, `CER = 0.1559`.

8.2. Качество на dev

Из файла `outputs/dev_run/metrics.json`:

- число примеров: 2265;
- итоговый `CER = 0.149639`.

8.3. Наблюдения по устойчивости

Качество зависит от формата и спикера:

- exact match на wav: 0.6906;
- exact match на mp3: 0.5349.

Это указывает, что MP3-компрессия и связанные с ней искажения заметно осложняют задачу.

Разбивка по спикерам на dev показывает, что модель хуже всего работает на:

- spk_K: 0.5131;
- spk_I: 0.5433;
- spk_C: 0.6100.

Лучшие результаты получены на:

- spk_B: 0.7400;
- spk_E: 0.7300;
- spk_D: 0.7200.

Такое распределение согласуется с постановкой соревнования: основной риск связан не с самим словарем числительных, а с переносом на unseen или более сложные голоса и шумные условия.

9. Submissions history

1. kaggle_submission_notebook - Version 1 Complete · Азат Резитдинов · 26.04 · Notebook kaggle_submission_notebook | Version 1 2.383
2. submission.csv
Complete · Sergei Trifonov · 23.04
2.369
3. submission.csv
Complete · Sergei Trifonov · 22.04
9.396
4. submission.csv
Error · Sergei Trifonov · 22.04

5. submission.csv

Complete · Sergei Trifonov · 19.04 · Conformer
11.711

10. Анализ ошибок

По `dev_predictions.csv` можно выделить несколько типичных режимов ошибки:

1. Потеря части числа:

- 967653 -> 13653
- 552318 -> 1000

2. Сведение сложного числа к более простой структуре:

- 675067 -> 600000
- 375543 -> 300040

3. Ошибки на последних разрядах:

- 138042 -> 138040
- 703604 -> 703600
- 748240 -> 748000

4. Замена соседних по звучанию фрагментов:

- 305874 -> 306874
- 275423 -> 275523

Основные выводы по ошибкам:

- модель в целом хорошо улавливает грубую структуру числа;
- больше всего страдают младшие разряды и редкие фонетически близкие фрагменты;
- часть ошибок возникает на MP3-файлах, что подтверждает важность устойчивости к деградациям сигнала;

- в текущем виде декодер не использует языковую модель, поэтому некоторые структурно неверные фразы не исправляются на этапе поиска.

11. Что можно улучшить

Наиболее реалистичные направления улучшения:

- усилить шумовые аугментации и отдельно таргетировать MP3-like degradations;
- ввести speaker-aware анализ CER во время обучения и early stopping по более устойчивой метрике;
- улучшить обратный парсер `words_to_num()` и валидацию допустимых словесных шаблонов;
- попробовать более сильный decoding с ограничениями на грамматику русских числительных.

12. Полезные ссылки

- [Group Project 1. Automatic Speech Recognition - \[30 pts\]](#)